3 Model-View-Controller (MVC)

Model-View-Controller or the MVC is a common paradigm used to logically divide the code that builds up your GUI (Graphical User Interface). The MVC model of Cocoa Touch divides up the functionality into Model: The classes or objects that hold your data; View: The front panel of your code made up of windows, the area where you user actually interacts with your code; and Controller: Binds the model and view together and is the application logic that decides how to handle the user's inputs. Controller though, can be customised. They are typically subclasses of one of the many generic controller classes from the UIKit framework. An example of this is the UIViewController.

MVC methodology also emphasises strongly upon the reuse of code, to minimise human efforts.

If you again focus on the "Groups and Files" pane in your Xcode window you will notice that it contains of four set of files (let us assume your project is called digit):

```
digitAppDelegate.h
digitAppDelegate.m
digitViewController.h
digitViewController.m
```

The .h files are the header files and .m files are the main source code files. As you would have easily guessed, the third and the fourth files are the ones controlling the ViewController. Let's look at some of the lines of code in our introductory program.

```
#import   <  UIKit/UIKit.h  >
@interface digitViewController : UIViewController {
}
-(IBAction) bnClicked:(id)sender; //Ignore this
@end
```

The first two lines declare your ViewController to be a subclass of the genereic UI View Controller that is available in the UIKit/UIkit.h, and gives us basic functionality in a ready to go manner.

`@end` is simply a marker defining the end of the code.

Till now, we have already seen how to modify any attribute of any object in your view window by using the inspector functionality via GUI. If you want to do the same with code, you will need to create an "outlet". An outlet